A source-specific research and implementation note

Five Further Lean Proof Simplifications

for `DiophantineConsistency`

Prepared for Vladimir Reshetnikov

26 October 2026

Source revision. `9d9d1123b39927d30363f1dcd53c23f0575c30ee`.
Compiler baseline. Lean 4.32.0; Mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`.

Abstract

This iteration treats every proposal in the two requested exclusion locations as available infrastructure, whether adopted or pending. It identifies five additional changes at six declarations: exact exponent normalization for the raw cyclic digit rotation; a stronger two-summand capacity bound for nonwrapping; internal quotient uniqueness by cancelling the common quotient contribution; zero-test soundness derived from its product and reconstruction equations; and direct linear arithmetic for the fixed controller progression. Five independent Lean files check the mathematical mechanisms, including copied production definitions where relevant. Four patch files propose edits to five production modules. Their complete-file line counts decrease from 783 to 763, charging the additional import in the zero-test module. This is a measured textual delta of the proposed patches, not a completed project integration or a build-time claim. The most useful gains are weaker proof dependencies and stronger intermediate statements; the immediate line savings are deliberately modest.

1 Outcome and scope

The strongest conceptual changes are the first, second, and fourth proposals. They replace an unnecessarily specialized intermediate statement with the fact actually needed by the client. Rotation equality depends on the phase and cyclic shift, so it can be established before proving anything about the digit values. Nonwrapping follows from the two units of slack in two strict natural-number bounds, so a quadratic radix inequality is unnecessary. The zero-test output follows from two of its certificate equations, so soundness need not first classify roots of the Boolean equation.

The division proposal simplifies a proof inside an arbitrary internal natural-number structure. It cancels a common summand and uses one positive quotient gap. Its value is that it removes the second slack witness and the constructed self-inequality while retaining the intended nonstandard-model semantics. The controller proposal is a smaller cleanup: its apparently nonlinear multiplication has a fixed natural coefficient, and its exact affine form is suitable for `omega`.

All public theorem statements in the proposed patches retain their existing binders and conclusions. In particular, the immediate rotation patch retains the original Boolean premise, although the new proof does not use it. The zero-test patch concerns the existing three-row certificate and does not alter the literal polynomial system. The patch for that file adds an import exposing the existing ordered-semiring instance, and that line is charged in the table.

1.1 What has actually been checked

The independent Lean prototypes establish the mathematical cores against the pinned compiler and Mathlib revisions. The rotation file copies the production definitions of `rotate`, `shiftDigits`, and

ID	Target and mathematical change	File lines	Immediate value
R	`CurrentWordRephase2011.rotation_eq`: compare exponent observations directly	122 → 119	3 fewer lines; Boolean reasoning removed from body
C	`CurrentDummySafety2011.nonwrap_of_short`: two strict bounds in the same window	213 → 208	5 fewer lines; stronger reusable bound
D	`ObjectDivision.quotient_not_lt`: cancel qd, retain one gap	179 → 175	4 fewer lines; one fewer witness
Z	`ObjectZeroTest.certificate_sound`: product and reconstruction rows	91 → 89	2 fewer lines after one import
A	`CurrentControllerPositions2011`: injectivity and exclusion of 3	178 → 172	6 fewer lines; linear arithmetic
	Total, five proposed production files	**783 → 763**	**20 fewer lines**

Table 1: Physical line counts of complete source files. Optional general lemmas and the independent research prototypes are not inserted by these patches.

`binaryRotation`; the controller file copies the exact controller constant, position formula, and `ZMod` domains. The two internal-arithmetic files work over abstract algebraic carriers, so their proofs do not identify an internal model with the external natural numbers. The capacity file is a direct proof over `Nat`, including the exact statement of the current private lemma.

These checks do not rebuild the complete production dependency graph. Thus the patches are concrete, reviewable integration proposals. The archive supplies compiler logs, axiom output, source hashes, patch checks, and portable reproduction commands. It contains no claim that the 20-line textual reduction has already passed the repository's complete integration gates.

2 Exclusion audit: earlier ideas are the baseline

The audit is pinned to the same revision as the source review. It covers the placed proof-simplification directory and all five archives in `docs/incoming`. There are 36 dated placed packets, from `2026-10-26a` through `2026-10-26zj`. Across the two requested locations the tracked-path census contains 2,175 paths. Text extraction, including text members of ZIPs, yields 3,768 entries and 2,874 distinct text hashes. These include repeated source snapshots and nested copies of earlier reports; they are not counts of distinct ideas. The direct report corpus contains 127 component files, about 1.56 MB of prose and report source.

The procedure was to inventory the complete text corpus, scan the report titles and section structure, and compare each candidate against the closest report mechanisms and earlier candidate source variants. Source snapshots were distinguished from proposed edits. A report that merely bundles an unchanged module does not exhaust every possible later simplification of that module. Conversely, a proposal counts as excluded even if it was never implemented, was left as future work, or has no successful compilation record.

The following families are therefore treated as already supplied: assignment naturality and witness layouts; generic sum-of-squares and coefficient envelopes; compiler schema generation; PR composition and scanner invariants; internal arithmetic transport and successor discreteness; signed Pell residue interfaces; rotation via convolution with an impulse; degree-before-coefficient reasoning; finite-support and helical-period interfaces; coherent carry records and reset-local carry arguments; shared modular

kernel certificates; and finite weighted point updates. None is claimed again as a contribution of this report.

ID	Closest earlier mechanism	Additional content retained here
R	packet i: `word_rotate` from impulse convolution; earlier phase estimates	Exact equality of the raw digit function after reducing the bit exponent modulo the complete bit window. It applies to unbounded digits and specializes to a fresh rephasing client.
C	Earlier power cancellation; incoming mutation packets retaining a nonwrap guard	A stronger sufficient capacity statement with two summands under the same boundary, including the radix-two endpoint. The old guard is discharged through this specific inequality.
D	packet k: external natural quotient certificates; packet zf: internal division existence	An additional simplification of internal quotient uniqueness: cancel the common qd, eliminating the remainder-gap witness.
Z	packet x: Boolean idempotence by cancellation; packet m: consume Boolean certificates; packet zg: identity transport	Eliminate the Boolean equation from the soundness dependency set itself. The product and reconstruction rows determine the selector.
A	packet zh and packet zj: physical-order and finite-order interfaces	Resolve two new, literal affine-arithmetic obligations at coefficient 120198, with a six-line local reduction.

The archive's `evidence/novelty-notes.md` gives a packet-by-packet map, including the five incoming archives. `novelty-matrix.csv` records the closest precedent for each retained idea. The conclusion is relative novelty within the inspected, pinned corpus. It is not a claim of mathematical originality for elementary cancellation or quotient/remainder identities, nor an absolute guarantee against a differently phrased equivalent sentence in a large archive.

3 R: normalize the exponent of the raw digit rotation

3.1 The current proof uses a stronger interpretation than equality requires

The production definition in `NativeCyclicRotation.lean` is

```
def binaryRotation (v W : ℕ) (c : ZMod n → ℕ) : ZMod n → ℕ :=
  rotate (shiftDigits (2 ^ v) (2 ^ (W % v)) c)
    ((W / v : ℕ) : ZMod n)
```

where

$$\mathrm{shiftDigits}(\rho, s, c)(k) = (sc(k)) \bmod \rho + \left\lfloor \frac{sc(k-1)}{\rho} \right\rfloor, \qquad \mathrm{rotate}(c, t)(k) = c(k-t).$$

This definition retains the quotient contribution from the preceding digit. It is meaningful even when the input digits are not Boolean or smaller than the radix.

The client `CurrentWordRephase2011.rotation_eq` compares displacements $47(2011s)$ and $47(2011t)$ when the cell strides agree in $\mathbb{Z}/N\mathbb{Z}$. Its current proof invokes the Boolean phase-zero theorem twice

and then compares the resulting pointwise permutations. But the two calls to `binaryRotation` already have the same partial phase and the same cyclic whole-digit shift. Equality can be proved at that level before interpreting any output digit.

3.2 A stronger exact statement

Theorem 3.1 (Exponent normalization)**.** *For all natural v, n, W and all functions $c : \mathrm{ZMod}(n) \to \mathbb{N}$,*

$$\mathrm{binaryRotation}(v, W \bmod (vn), c) = \mathrm{binaryRotation}(v, W, c).$$

The equality is an equality of functions.

Proof. If $v = 0$ or $n = 0$, then $W \bmod (vn) = W$, so the conclusion is immediate. Otherwise the exponent enters the definition only through $W \bmod v$ and the class of $\lfloor W/v \rfloor$ modulo n. Natural division and remainder satisfy

$$(W \bmod (vn)) \bmod v = W \bmod v, \qquad \left\lfloor \frac{W \bmod (vn)}{v} \right\rfloor = \left\lfloor \frac{W}{v} \right\rfloor \bmod n.$$

Casting the right-hand remainder to `ZMod n` gives the same class as casting the original quotient. Thus every argument to `rotate` and `shiftDigits` is unchanged. □

The checked Lean proof is only the corresponding simplification:

```
theorem binaryRotation_mod (v W : ℕ) (c : ZMod n → ℕ) :
    binaryRotation v (W % (v * n)) c = binaryRotation v W c := by
  simp only [binaryRotation, Nat.mod_mul_right_mod,
    Nat.mod_mul_right_div_self, ZMod.natCast_mod]
```

Consequently congruent exponents modulo vn produce equal digit functions. For positive v and n this gives an actual period; the degenerate zero cases should be described as normalization identities, not as positive periodicity.

3.3 Immediate implementation and accounting

The existing whole-cell client can use its already proved `physical_shift_eq` directly:

```
  simp only [binaryRotation, displacement, Nat.mul_mod_right,
    Nat.mul_div_cancel_left _ (by decide : 0 < 47),
    physical_shift_eq s t hst]
```

The supplied patch formats this as two proof lines, replacing five existing lines. It adds no import and no helper. The original `hc` premise remains in the theorem header for compatibility and becomes unused; a later API cleanup may expose an unrestricted theorem while retaining the original wrapper. That later wrapper change is not included in the measured saving.

The general normalization theorem is worth adding when additional clients compare arbitrary partial phases or exponents presented in different arithmetic forms. One current two-line client does not by itself amortize a new abstraction layer. This distinction matters: the elementary local patch saves three lines, while the optional stronger API adds declarations and provides a better statement for future proofs.

3.4 Boundary cases and validation

`lean/RotationPeriod.lean` checks the exact copied definitions, the normalization theorem, its congruent-exponent corollary, a whole-digit equality theorem, and the literal 47/2011 specialization. It also elaborates zero-width, zero-cycle, and one-cell examples. The generic theorems require no digit upper bound. There is no assumption that either ordinary multiplier 2^W agrees with another multiplier as a natural number: the conclusion concerns the cyclic digit operation. This is precisely the distinction needed by the rephasing module, whose scalar quotient witnesses may change.

4 C: obtain nonwrapping from two units of strict slack

4.1 The current arithmetic detour

The private theorem `CurrentDummySafety2011.nonwrap_of_short` assumes

$$\rho \geq 2, \quad m \geq 2, \quad F < \rho^{m-1}, \quad \delta < \rho^{m-2}$$

and proves $F + \delta < \rho^m - 1$. Its current proof factors through ρ^{m-2}, proves $\rho + 1 < \rho^2$, and combines several polynomial inequalities. The destination modulus is one less than the full radix power, so a proof must account for strict natural-number slack rather than merely establish $F + \delta < \rho^m$.

4.2 A stronger capacity theorem

Theorem 4.1 (Two strict summands in one window)**.** *Let $\rho, m, F, \delta \in \mathbb{N}$, with $\rho \geq 2$ and $m \geq 1$. If*

$$F < \rho^{m-1} \quad \textit{and} \quad \delta < \rho^{m-1},$$

then $F + \delta < \rho^m - 1$.

Proof. Put $B = \rho^{m-1}$. The two strict bounds give $F + 1 \leq B$ and $\delta + 1 \leq B$. Hence

$$F + \delta + 2 \leq 2B \leq \rho B = \rho^m.$$

This is exactly the integer slack required for $F + \delta < \rho^m - 1$. □

The proof first isolates an exponent-free capacity fact:

```
theorem two_slots {F δ B Q : ℕ} (hF : F < B) (hδ : δ < B)
    (hcapacity : B + B ≤ Q) : F + δ < Q - 1 := by
  omega
```

Only one multiplicative inequality is then needed to instantiate it:

```
theorem same_window {ρ m F δ : ℕ} (hρ : 2 ≤ ρ) (hm : 1 ≤ m)
    (hF : F < ρ ^ (m - 1)) (hδ : δ < ρ ^ (m - 1)) :
    F + δ < ρ ^ m - 1 := by
  have hcapacity := Nat.mul_le_mul_right (ρ ^ (m - 1)) hρ
  have he : ρ * ρ ^ (m - 1) = ρ ^ m := by
    rw [← pow_succ']
    congr 1
    omega
  rw [he] at hcapacity
  exact two_slots hF hδ (by omega)
```

The present lemma follows because $m-2 \le m-1$ and powers are monotone for $\rho \ge 1$. The inline patch applies the same argument without adding the two general declarations to production. It removes five lines from the complete module and uses no new imports.

4.3 Why this is a useful strengthening

The second summand may now occupy the same penultimate window as the first. The current client has a stronger hypothesis on δ, so its statement and behavior are preserved by the patch. A later client can use the more permissive theorem if its actual increment bound is only $\delta < \rho^{m-1}$. That is a genuine improvement of the sufficient condition, not a restatement of a previously assumed nonwrap guard.

The endpoint $\rho = 2$ is instructive. At $m = 3$, both summands may equal 3, giving $6 < 7$. Replacing one strict bound by a weak bound would allow $3 + 4 = 7$, which violates the conclusion. Thus the two units of slack are doing essential work. The condition $m \ge 1$ is also used: it justifies $\rho\,\rho^{m-1} = \rho^m$ despite natural truncated subtraction. All these issues are visible in the stronger statement rather than hidden in a large arithmetic tactic call.

This proposal does not assert any additional preservation property for masks or changed words. It supplies the numeric nonwrap fact used by the existing mutation reasoning. The earlier mutation, rank, and carry infrastructure remains the assumed baseline.

5 D: cancel the common quotient contribution in internal division

5.1 The exact source obligation

In `ObjectDivision.lean`, the carrier is `Natural H`, the internal omega of an arbitrary model satisfying the relevant set-theoretic axioms. The relation is

$$\mathrm{DivisionRel}(q, r, n, d) \quad \Longleftrightarrow \quad n = qd + r \ \wedge \ r < d.$$

The private theorem `quotient_not_lt` takes two such representations and proves $\neg(q < q')$. The present proof chooses both a quotient gap and a remainder gap, then builds a positive slack witnessing the impossible inequality $n < n$.

The division-existence proof uses internal induction and has already been the subject of earlier proposals. The change here is confined to the uniqueness argument after two representations have been supplied.

5.2 One gap is enough

Assume $q < q'$. The existing discrete-order theorem supplies s with

$$q' = q + s + 1.$$

The two representation equations imply

$$qd + r = q'd + r' = (q + s + 1)d + r' = qd + (sd + d + r').$$

Additive cancellation gives $r = sd + d + r'$. Canonical additive order now gives $d \le sd + d + r' = r$, contradicting $r < d$. No remainder-gap witness is needed.

The proposed proof body is:

```
intro hq
obtain ⟨s,hs⟩ :=
  (ObjectNaturalDiscrete.lt_iff_exists_add_one H q q').mp hq
have he : r=s*d+d+r' := add_left_cancel (a := q*d)
  (h.1.symm.trans (h'.1.trans (by rw [hs]; ring)))
exact not_le_of_gt h.2
  (he.symm ▷ (le_add_self.trans le_self_add))
```

The patch uses compact formatting for the last two expressions. The complete source file decreases from 179 to 175 lines, without helpers or imports.

5.3 The important semantic boundary

The proof does not represent an internal natural as an external `Nat`, invoke external well-founded induction, or assume an Archimedean property. Multiplication monotonicity is unnecessary too: after cancelling qd, the inequality $d \le sd + d + r'$ follows entirely from canonical additive order. This makes the proof applicable on the entire internal omega, including its nonstandard elements.

The independent theorem in `lean/InternalQuotientGap.lean` is checked over a commutative semiring with a linear order, canonical additive order, and additive cancellation. Its only discreteness input is the explicit implication

$$a < b \implies \exists s,\ b = a + s + 1.$$

The project already has this implication. The abstract proof therefore separates the algebraic cancellation mechanism from the previously established internal-model facts without turning those facts into hidden new assumptions.

The proof of $\neg(q < q')$ does not need the second remainder bound $r' < d$, although the public relation supplies it. The symmetric application in `division_unique` uses the corresponding bound on the other side. This observation explains the structure of the existing uniqueness theorem and avoids introducing an unnecessarily symmetric low-level helper.

6 Z: derive the zero-test selector from two certificate equations

6.1 A dependency reduction inside the existing certificate

The certificate in `ObjectZeroTest.lean` consists of

$$b^2 = b, \qquad bn = 0, \qquad n + b(s+1) = s + 1. \tag{1}$$

Its current soundness proof first uses $b^2 = b$ to split $b = 0$ or $b = 1$, then derives the corresponding statement about n. A more direct proof splits on the observed input $n = 0$ and uses the last two equations to determine b.

Theorem 6.1 (Two-row zero-test soundness)**.** *Let R be a commutative semiring with cancellation by nonzero factors. Suppose $s + 1 \neq 0$, $bn = 0$, and $n + b(s+1) = s + 1$. Then*

$$b = \begin{cases} 1, & n = 0, \\ 0, & n \neq 0. \end{cases}$$

Proof. If $n = 0$, the reconstruction equation becomes $b(s+1) = 1(s+1)$; cancel the nonzero factor $s+1$. If $n \neq 0$, cancel n from $bn = 0n$, equivalently use the zero-product property, to obtain $b = 0$. □

In Lean the minimal proof is:

```
theorem zeroTest_two_rows {bit n slack : R}
    (hs : slack+1 ≠ 0) (hbn : bit*n = 0)
    (hns : n+bit*(slack+1) = slack+1) :
    bit = if n = 0 then 1 else 0 := by
  by_cases hn : n = 0
  · simp only [hn, zero_add] at hns
    simpa only [if_pos hn] using
      mul_right_cancel₀ hs (hns.trans (one_mul _).symm)
  · simpa only [if_neg hn] using
      (mul_eq_zero.mp hbn).resolve_right hn
```

Here the class assumptions are `CommSemiring R`, `IsCancelMulZero R`, and `DecidableEq R`. The second theorem in the prototype uses the ordered-carrier assumptions relevant to the actual internal naturals and retains all three conjuncts of the source certificate. Thus the instance inference needed for that algebraic setting is also checked independently.

6.2 Production port and exact cost

The production proof calls `ObjectNaturalDiscrete.add_one_ne_zero H slack` for the nonzero successor. Its proposed extra import is `ObjectNaturalOrderedSemiring`, which exposes the already established strict ordered-semiring structure and the resulting nonzero multiplication cancellation. The import graph is acyclic: that file depends on `ObjectNaturalDiscrete`, not on the zero-test module.

The old proof block has 11 physical lines and the proposed block has 8. After charging the additional import, the complete file decreases from 91 to 89 lines. If the earlier multiplication-order proposal is already integrated so that the instance is exposed transitively, the import may be redundant; this report uses the conservative current-file accounting.

The result is more informative than the two-line net saving: the soundness proof has a smaller dependency set. Booleanity becomes a consequence of the two determining equations in this carrier. This is distinct from shortening the general theorem that classifies idempotent elements, and distinct from consuming an already supplied Boolean certificate more efficiently.

6.3 Why the hypotheses and literal rows must be distinguished

Nonzero cancellation matters. In a ring, $s = -1$ makes $s + 1 = 0$; with $n = 0$, the last two equations then hold for arbitrary b. The conclusion fails without a nonzero-successor premise. The internal-natural theorem has exactly the fact needed to exclude this degeneration.

The proposed patch leaves (1), `zeroRows`, the degree theorem, and the expression semantics as written. Deleting a polynomial row would be a different change: it could affect literal systems, cost statements, and certificates over other domains. The current contribution is a proof simplification for the existing certificate. The derivation also offers a useful review principle for future gadgets: first ask which rows determine the output on the intended carrier, then use that dependency information to organize soundness proofs. Any later alteration of a literal system needs its own analysis.

7 A: expose the fixed affine controller progression

The controller data defines `auxiliary` to be 120199. The position map in `CurrentControllerPositions2011.lean` is therefore

$$\text{position}(k, j) = 2 + 120198\, j.\,\text{val}, \qquad j : \text{ZMod}(120199^k).$$

Two current proofs use nonlinear arithmetic: injectivity of this map and exclusion of the optional position 3. Both obligations are actually linear over the natural representative because the coefficient is fixed.

The proposed exclusion proof is:

```
theorem position_ne_three (k : ℕ)
    (j : ZMod (auxiliary^k)) : position k j ≠ 3 := by
  change 2 + 120198 * j.val ≠ 3
  omega
```

The equation would require $120198\, j.\,\text{val} = 1$, which has no natural solution. No split on whether the representative is zero is needed.

Similarly, the injection proof is:

```
theorem position_injective (k : ℕ) :
    Function.Injective (position k) := by
  intro i j hij
  apply ZMod.val_injective
  change 2 + 120198 * i.val = 2 + 120198 * j.val at hij
  omega
```

This exposes the affine equality precisely at the arithmetic interface while retaining the existing `ZMod` representatives. An experiment that globally unfolded `auxiliary` changed the presentation of the representative expressions enough to confuse the arithmetic atoms; the delivered proof uses the explicit local `change` shown above.

The exact-domain pilot compiles both proofs with the real position and cell definitions. The combined patch removes six lines from the complete 178-line module and introduces no helper or import. It also applies at $k = 0$, when the source cycle has one element. Nothing in the proof needs $k > 0$.

This is intentionally a small cleanup. A new general affine-embedding framework would add overhead and repeat prior infrastructure proposals. If the controller parameter later becomes variable, a symbolic injectivity argument using positivity and an exclusion argument modulo the step size would be more appropriate. For the present literal controller, the two direct proofs are the smallest clear change.

8 Validation, adoption order, and limits

8.1 Compiler evidence

The archive contains five standalone files and thirteen named audited theorem declarations:

- `RotationPeriod.lean`: four theorems, plus zero-width, zero-cycle, and single-cell examples.
- `Capacity.lean`: four theorems, including both a reusable stronger theorem and an inline exact-statement version; three endpoint examples.

- `InternalQuotientGap.lean`: the abstract one-gap quotient argument.
- `TwoRowZeroTest.lean`: a minimal cancellation theorem and an ordered-carrier certificate theorem.
- `AffinePositions.lean`: two exact-domain controller proofs.

The `#print axioms` outputs contain only subsets of `propext`, `Classical.choice`, and `Quot.sound`. The quotient-gap theorem uses only `propext`; no new axiom, placeholder, or external arithmetic certificate is introduced by the prototypes. These outputs concern the independent theorems, not an axiom audit of every imported production theorem.

The compiler is Lean 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The Mathlib revision equals the one in the source repository's `lake-manifest.json`. Read-only cached Mathlib objects were reused. No cached `DiophantineConsistency` object is used as evidence for the changed production files; the independent files import Mathlib and the core tactic modules directly.

8.2 What the measurements mean

The line table counts complete candidate modules, not selected proof interiors. It charges the zero-test import. The proposed production patches add no general helper declarations. The stronger rotation and capacity APIs live in the research prototypes and are described separately, so their extra source is not hidden in a claimed net reduction.

Physical line count is only one measure of simplification. The division patch removes a witness and a constructed contradiction certificate. The zero-test patch removes dependence on the Boolean-root classification theorem. The rotation patch removes two semantic rewrites. The capacity patch replaces a quadratic radix inequality with a single multiplication monotonicity fact. These are structural observations about the actual proofs. There are no measured wall-clock or memory improvements, and no such improvements are claimed.

8.3 Recommended integration sequence

Start with the controller, rotation, and capacity body-only patches. They have small dependency surfaces and introduce no new imports. Rebuild each touched module and its direct clients using the repository's normal workflow. The rotation proof can leave an unused-premise warning because its public Boolean premise is retained; decide explicitly whether the project's warning policy calls for a compatibility wrapper or a future API change.

Next integrate the division proof. Confirm that the existing canonical additive order and cancellation instances are visible through its current imports. Its independent proof uses those exact algebraic capabilities and does not require the multiplication-order import.

Finally integrate the zero-test proof with the explicit ordered-semiring import, or omit that import if the assumed earlier work already exposes the instance. Rebuild the zero-test module and the clients consuming its certificate and expression theorems. Since the literal rows are unchanged, the relevant regression question is statement and elaboration compatibility, not a new polynomial construction.

Only after these clients are rebuilt should the 20-line candidate delta be called an integrated reduction. Add the optional general rotation-normalization or capacity API when a second client makes its cost worthwhile. The compact patches are useful even if those general declarations remain research lemmas.

8.4 Corpus and time boundary

The reviewed source is the fixed commit displayed on the first page. New commits or newly uploaded proposals can change the novelty assessment. The archive records the exact hashes of the source targets and the exclusion inventory so a later iteration can compare against this result without relying on the moving `main` branch.

9 Artifact guide and reproduction

Path	Purpose
`report.tex`, `report.pdf`	Complete article and its compiled PDF.
`lean/`	Five independent, checked Lean prototypes with axiom commands.
`patches/`	Four unified patch files affecting five source modules.
`evidence/validation.json`	Final independent compilation results and validation scope.
`evidence/logs/`	Compiler output for every delivered prototype.
`evidence/source_manifest.json`	Exact source revision, content hashes, and source-file sizes.
`evidence/novelty-matrix.csv`	Proposal-by-proposal comparison with the closest earlier ideas.
`evidence/novelty-notes.md`	Full placed-packet and incoming-archive map.
`evidence/exclusion-corpus-manifest.json`	Hashed inventory of extracted text inputs and ZIP members.
`scripts/check_sources.py`	Check target source hashes before considering the patches.
`scripts/check_lean.sh`	Compile the independent files in the pinned Lake environment.
`scripts/build_pdf.sh`	Recompile the article with `latexmk`.

For a fresh independent proof environment, install the toolchain named by `lean-toolchain`, run `lake update`, obtain the Mathlib cache with `lake exe cache get`, then run `sh scripts/check_lean.sh`. The `lakefile.toml` pins the exact Mathlib commit rather than a moving branch. For source integration, first run the source-hash checker against a checkout of the pinned ProveIt commit, then inspect the patches and use `git apply -check` before applying them. The source checker intentionally fails on a different target hash; a changed revision needs a new comparison.

Primary sources

[1] ProveIt, reviewed commit `9d9d1123b399`; the source manifest in the archive records full hashes.

[2] `NativeCyclicRotation.lean` and `CurrentWordRephase2011.lean`: literal rotation definitions and the whole-cell rephasing client.

[3] `CurrentDummySafety2011.lean` and `CurrentWordTopBounds2011.lean`: existing nonwrap guard, increment bounds, and normalized-word upper bounds.

[4] `ObjectDivision.lean`: internal division existence and uniqueness, especially `quotient_not_lt`.

[5] `ObjectZeroTest.lean`, `ObjectNaturalOrderedSemiring.lean`, and `ObjectNaturalDiscrete.lean`: literal zero-test certificate and the internal arithmetic interfaces.

[6] `CurrentControllerData2011.lean` and `CurrentControllerPositions2011.lean`: the exact auxiliary constant and affine positions.

[7] Lean 4.32.0, `Init/Data/Nat/Mod.lean`: the natural quotient and remainder identities used in rotation normalization.

[8] Mathlib, pinned `Mathlib/Data/ZMod/Basic.lean`: cast normalization and representative injectivity.

[9] The 36 placed proposal packets and the five incoming ZIP archives, all at the reviewed commit. Their mechanisms are treated as already applied throughout this report.